

Object Oriented Programming

Java



What is Inheritance?

- Inheritance is the concept of a child class (sub class) automatically inheriting the variables and methods defined in its parent class (super class).
- A primary feature of object-oriented programming along with encapsulation and polymorphism.



Why Inheritance?

- **Benefits of Inheritance in OOP: Reusability**
 - Once a behavior (**method**) is defined in a super class, that behavior is automatically inherited by all subclasses.
 - Thus, you write a method only once and it can be used by all subclasses.
 - Once a set of properties (**fields**) are defined in a super class, the same set of properties are inherited by all subclasses.
 - A class and its children share common set of properties.
 - A subclass only needs to implement the differences between itself and the parent.



Derive Sub-Class

- To derive a child class, we use the **extends** keyword.
- Suppose we have a parent class called Person.

```
public class Person {  
    protected String name;  
    protected String address;  
    public Person(){  
        System.out.println("Inside Person:Constructor");  
        name = ""; address = "";  
    }  
}
```



Derive Sub-Class

- Now, we want to create another class named **Student**.
- Since a student is also a person, we decide to just extend the class Person, so that we can inherit all the properties and methods of the existing class Person.
- To do this, we write;

```
public class Student extends Person {  
    public Student(){  
        System.out.println("Inside Student:Constructor");  
    }  
}
```



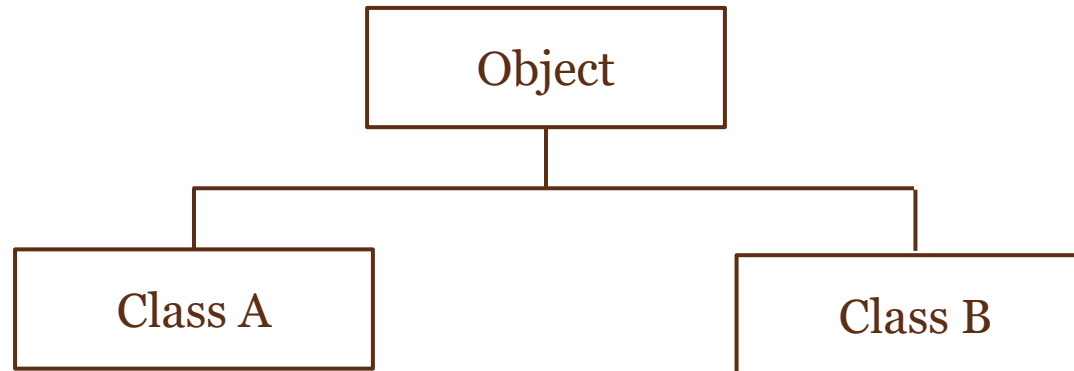
Privileges of Sub-Class

- A subclass inherits all of the “**public**” and “**protected**” members (fields or methods) of its parent, no matter what package the subclass is in.
 - A subclass **does not inherit the private members** of its parent class.
 - However, if the super class has public or protected methods for accessing its private fields, these can also be used by the subclass.
- The inherited fields can be used **directly**, just like any other fields.
- **New fields** can be declared in the subclass that are not in the super class.
- A field in the subclass with the same name as the one in the super class, thus hiding it (not recommended).



Object Class

- Object class is **mother** of all classes
 - In Java language, all classes are sub-classed (extended) from the Object super class
 - Object class is the only class that does not have a parent class
- Defines and implements behavior common to all classes including the ones that you write
 - getClass()
 - equals()
 - toString()
 - ...



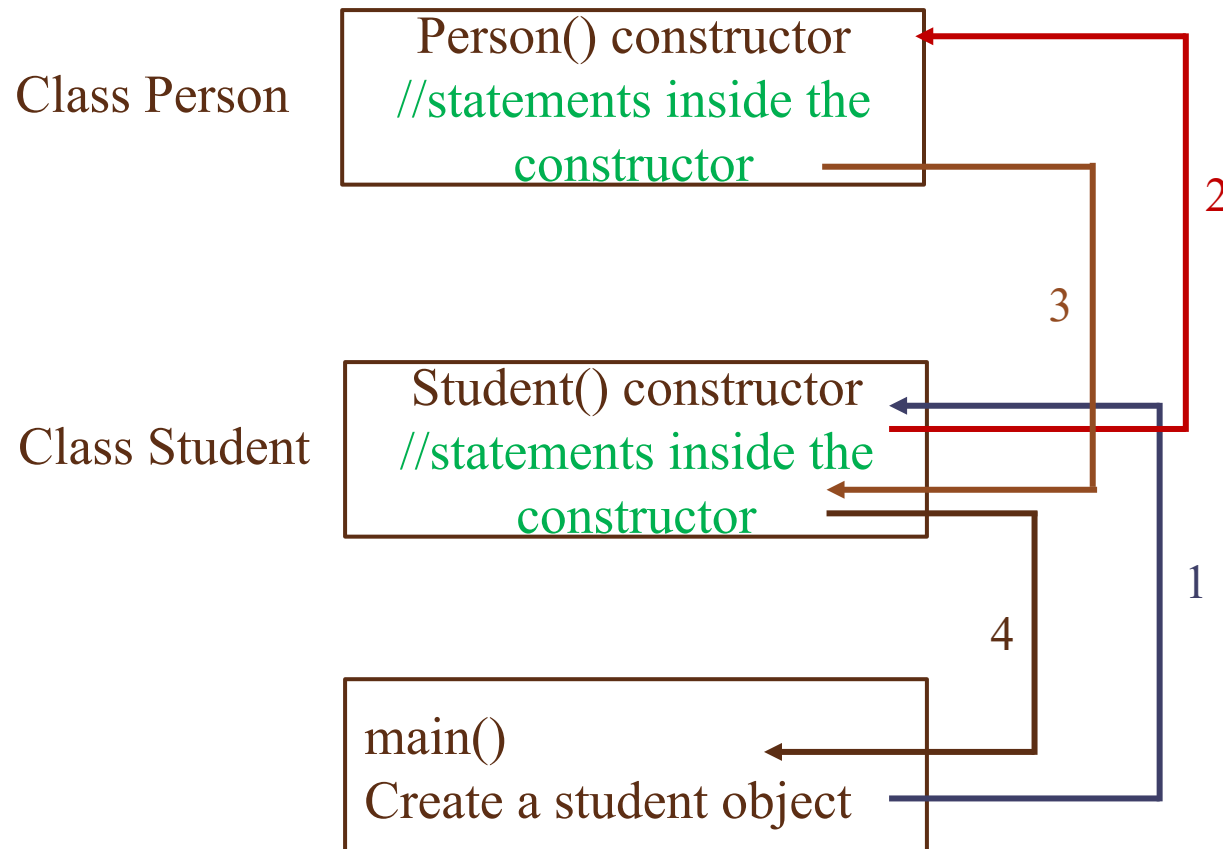
Constructor Calling Chain

- A subclass constructor invokes the constructor of the super class **implicitly**
 - When a Student object, a subclass (child class), is instantiated, the default constructor of its super class (parent class), Person class, is invoked ***implicitly before sub-class's constructor method is invoked***
- A subclass constructor can invoke the constructor of the super **explicitly** by using the “**super**” keyword
 - The constructor of the Student class can ***explicitly invoke the constructor of the Person class using “super” keyword***
 - Used when **passing parameters** to the constructor of the super class



Constructor Calling Chain

- Consider the following program flow:



Output:

Inside Person:Constructor
Inside Student:Constructor



‘super’ Keyword

- A subclass can also **explicitly call** a constructor of its immediate super class.
- This is done by using the **super** constructor call.
- A super constructor call in the constructor of a subclass will result in the execution of relevant constructor from the super class, based on the arguments passed.

```
public Student(){  
    super( "SomeName", "SomeAddress" );  
    System.out.println("Inside Student:Constructor");  
}
```



‘super’ Keyword-Rules

- The super() call **must occur** as the first statement in a constructor.
- The super() call can **only be used in a constructor** (not in ordinary methods).
- It is used to refer to members of the super class (just like the **this** reference).

```
public Student() {  
    super.name = "somename";  
    super.address = "some address";  
}
```



Overriding Methods

- If a derived class needs to have a different implementation of a certain instance method from that of the super class, that instance method can be overridden in the sub class
 - The scheme of overriding applies only to **instance methods**
- The overriding method has the **same name**, **number** and **type of parameters**, and **return type** as the method it overrides.



Overriding Methods (Example)

```
public class Person {  
    public String getName(){  
        System.out.println("Parent: getName");  
        return name;  
    }  
}  
  
public class Student extends Person{  
    public String getName(){  
        System.out.println("Student: getName");  
        return name;  
    }  
}
```

When we invoke the getName method of an object of the subclass Student, the getName method of the Student would be called, and the output would be

Student: getName

